

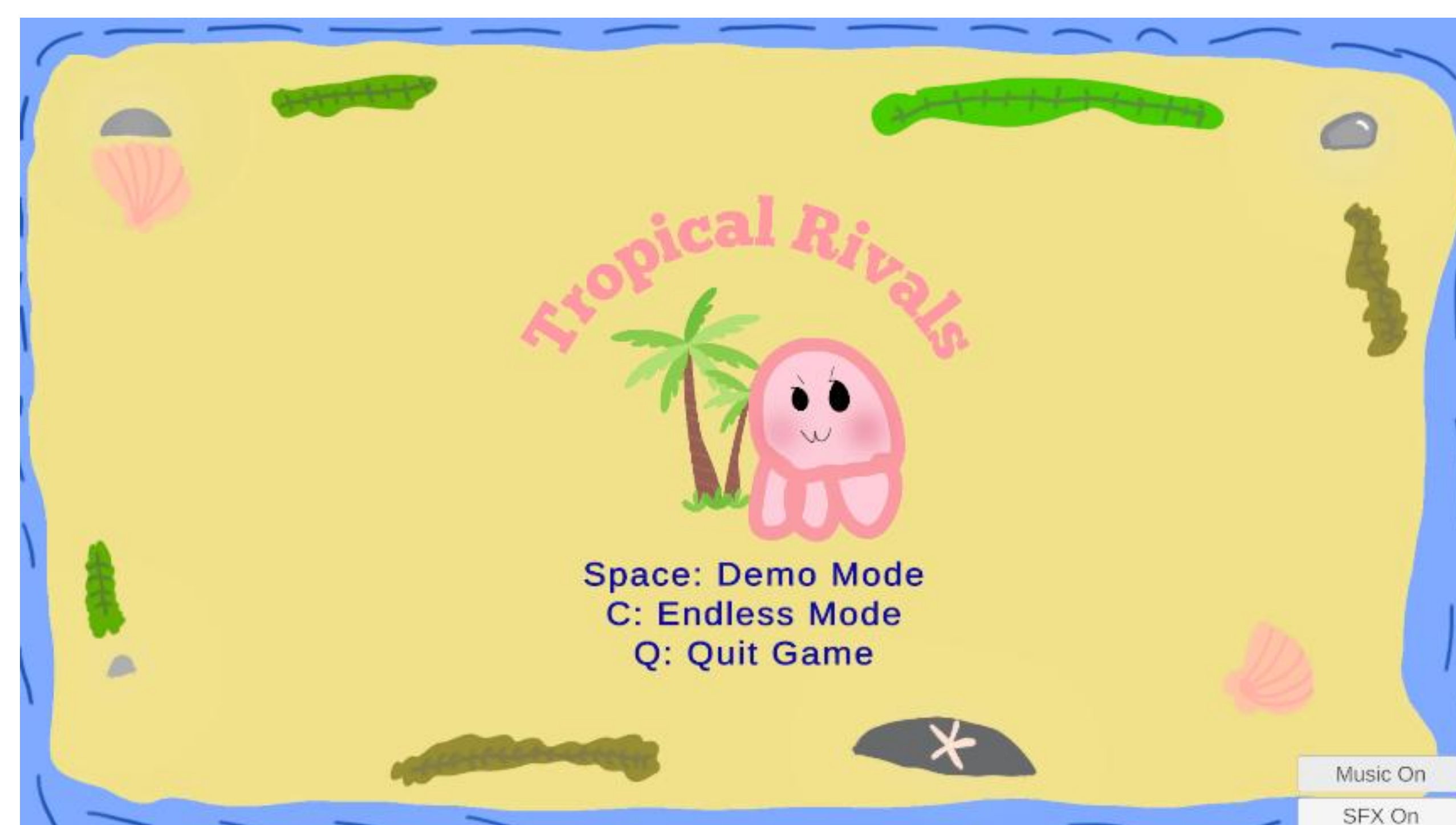


Tropical Rivals

Chris Hernandez • Professor Michael Soltys • COMP 499

Introduction

Tropical Rivals is a 2D roguelike game where players control an Octo and survive against waves of enemies in an oceanic region. The game includes a standard wave mode and an Endless Mode, where enemy stats gradually increase as the player clears more waves. This project focuses on creating a simple, replayable, and competitive gameplay experience that can be enjoyed casually or challenged at higher difficulty levels.



Methodology

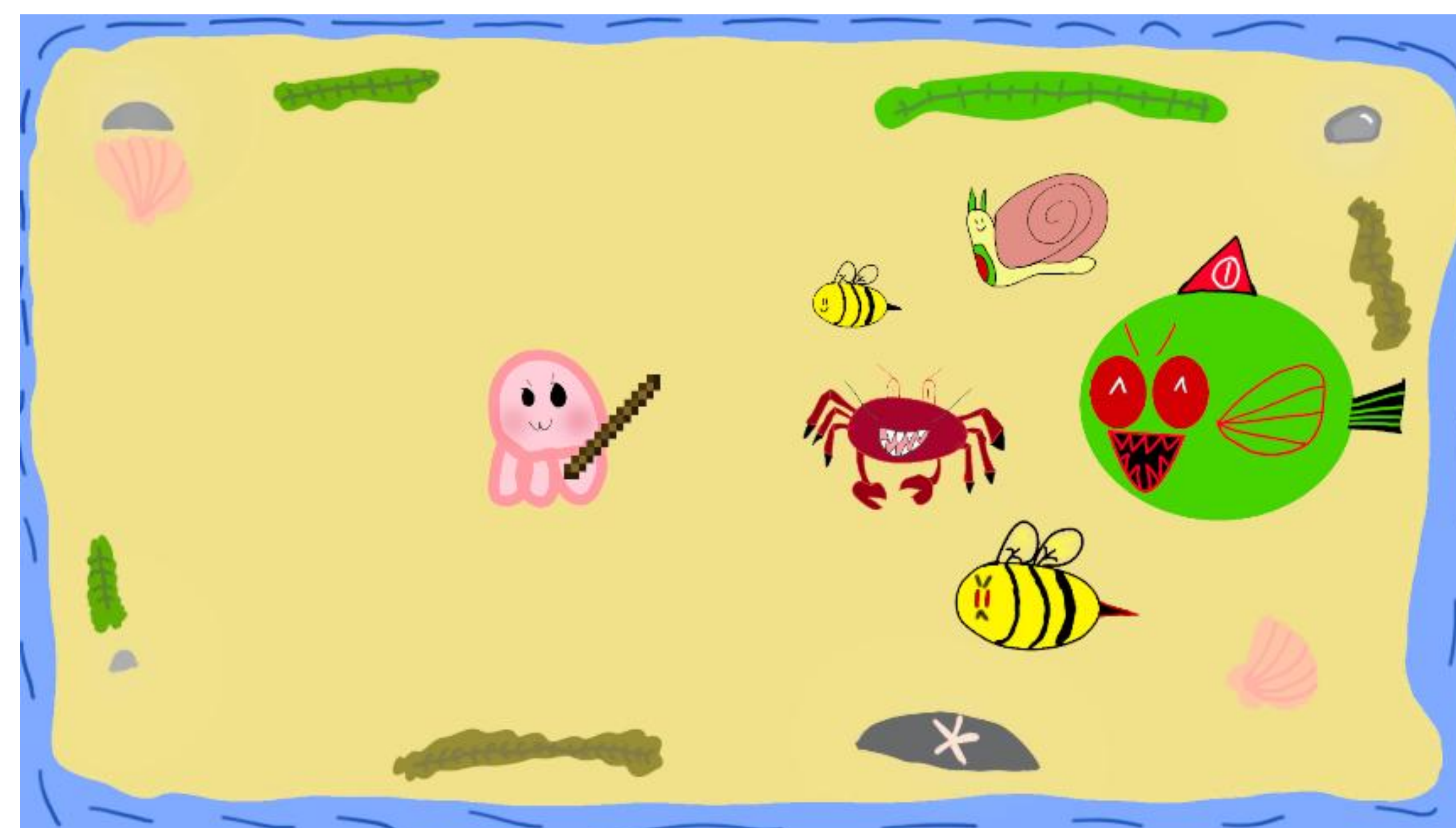
This whole development process has four major tasks.

- Brainstorming ideas, creating early game documentation, and exploring my project vision.
- Researching C# programming and Unity tools that will support my development process.
- Building a Unity prototype with simple player movement, enemy spawning, and core mechanics systems in place.
- Creating my assets for Octo, enemies, and the game environment.
- Playtesting, debugging, polishing, and adding QoL improvements.

Software Tools and Game Design

Software Design: The software I used to help build my project includes the Unity Game Engine, Microsoft Visual Studio, and Sketchbook. Unity was used to create the project and support the tools needed for my game design. Microsoft Visual Studio was used to write C# scripts that work with Unity, while Sketchbook was used to freely draw my assets and background.

Game Design: This game takes place on an oceanic island where Octo, the main character, must survive waves of enemies. These enemies swarm the island due to an unknown source, and Octo must clear each wave to survive. Some enemies may be possessed, while others may simply be hungry, curious or something else...



Future Improvements

Honestly, I enjoyed developing my game, but this was easily one of the most difficult projects I have ever worked on. I was also inexperienced with game design at the start, but this project helped me learn more about Unity, C#, and the overall development process.

Future Improvements:

- Add more playable characters, enemies, and items that can increase player stats or help defend the player.
- Improve the art design for some enemies and add more oceanic backgrounds.
- Implement a leaderboard system where players can submit their highest waves or scores.

Gameplay System

The gameplay focuses on Octo surviving waves of enemies, including Smol Bee, Sneaky Crab, Sea Snail, Possessed Pufferfish, and Angy Bee. Octo uses a stick to damage enemies, while enemies spawn randomly around the island and deal contact damage if they touch the player. Defeating enemies heals Octo based on the enemy type. The game begins with a 5-wave demo mode, then continues into Endless Mode where enemies gain small HP, ATK, and SPD increases each wave until Octo reaches zero HP and receives a final wave score.

Enemy Spawner & Endless Scaling Code

```
150 // Spawns an enemy around the player and applies Endless Mode scaling.
151 // In Endless Mode, the enemy's HP, damage, and speed increase based on the current wave.
152
153 void SpawnEnemy(GameObject prefab)
154 {
155     Vector2 randomOffset = Random.insideUnitCircle.normalized * Random.Range(minSpawnDistance, spawnRadius);
156     Vector3 spawnPos = player.position + new Vector3(randomOffset.x, randomOffset.y, 0f);
157
158     GameObject enemy = Instantiate(prefab, spawnPos, Quaternion.identity);
159
160     EnemyHealth health = enemy.GetComponent<EnemyHealth>();
161     if (health != null)
162     {
163         health.spawner = this;
164
165         if (waveManager != null && waveManager.endlessMode)
166         {
167             health.maxHP *= waveManager.GetHPMultiplier();
168             health.currentHP = health.maxHP;
169         }
170     }
171
172     EnemyDamage damage = enemy.GetComponent<EnemyDamage>();
173     if (damage != null && waveManager != null && waveManager.endlessMode)
174     {
175         damage.damage *= waveManager.GetATKMultiplier();
176     }
177
178     EnemyFollow follow = enemy.GetComponent<EnemyFollow>();
179     if (follow != null)
180     {
181         follow.player = player;
182
183         if (waveManager != null && waveManager.endlessMode)
184         {
185             follow.speed *= waveManager.GetSpeedMultiplier();
186         }
187     }
188
189     // Scales enemy HP, attack, and speed based on the current Endless Mode wave.
190     public float GetHPMultiplier()
191     {
192         return 1f + ((currentWave - 1) * 0.05f);
193     }
194
195     public float GetATKMultiplier()
196     {
197         return 1f + ((currentWave - 1) * 0.02f);
198     }
199
200     public float GetSpeedMultiplier()
201     {
202         return 1f + ((currentWave - 1) * 0.01f);
203     }
204 }
```

Acknowledgements

Thank you to Dr. Soltys for his guidance and support throughout this capstone project. I also want to thank my Esports and Computer Science friends and colleagues for their support, feedbacks, and encouragement during development. Lastly, a special thank to my father for encouraging me to pursue Computer Science and for motivating me to create a video game of my own.